



Security Assessment Report

Orca Whirlpools

PRs 1392 and 1396

September 30, 2026

Summary

The Sec3 team was engaged to conduct an incremental review of PRs 1392 and 1396 of the Orca Whirlpools.

The artifact of the audit was the source code of the following programs, excluding tests, in <https://github.com/orca-so/whirlpools>.

The initial audit focused on the following versions and revealed 1 issues or questions.

#	Task	Type	Commit
P1	PR1392 - Prepare for SIMD-0460 Virtual Address Space Adjustments	Solana	59f4c4c → b823c8d
P2	PR1396 - Improve checkpoint update	Solana	90333c1 → 5207a03

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [P2-I-01] Frequent updates reduce reward accrual 4

Appendix: Methodology and Scope of Work 6

Result Overview

Issue	Impact	Status
PR1392: PREPARE FOR SIMD-0460 VIRTUAL ADDRESS SPACE ADJUSTMENTS		
No issues found		
PR1396: IMPROVE CHECKPOINT UPDATE		
[P2-I-01] Frequent updates reduce reward accrual	Info	Acknowledged

Findings in Detail

PR1396: IMPROVE CHECKPOINT UPDATE

[P2 -I -01] Frequent updates reduce reward accrual

Identified in commit [5207a03](#).

Rewards accrue from elapsed time, `emissions_per_second_x64`, and active liquidity. `update_fees_and_rewards()` calls `calculate_fee_and_reward_growths()`, `_calculate_modify_liquidity()`, `next_whirlpool_reward_infos()`, then advances `reward_last_updated_timestamp`:

```
/* programs/whirlpool/src/instructions/update_fees_and_rewards.rs */
023 | pub fn handler(ctx: Context<UpdateFeesAndRewards>) -> Result<()> {
032 |     let (position_update, reward_infos) = calculate_fee_and_reward_growths(
033 |         whirlpool,
034 |         position,
035 |         lower_tick_array.deref(),
036 |         upper_tick_array.deref(),
037 |         timestamp,
038 |     )?;
040 |     whirlpool.update_rewards(reward_infos, timestamp);
```

Each update adds `floor(time_delta * emissions_per_second_x64 / liquidity)` to global reward growth. The division remainder is discarded, and advancing the timestamp prevents it from contributing to later updates:

```
/* programs/whirlpool/src/manager/whirlpool_manager.rs */
007 | pub fn next_whirlpool_reward_infos(
008 |     whirlpool: &Whirlpool,
009 |     next_timestamp: u64,
010 | ) -> Result<[WhirlpoolRewardInfo; NUM_REWARDS], ErrorCode> {
011 |     let curr_timestamp = whirlpool.reward_last_updated_timestamp;
023 |     let time_delta = u128::from(next_timestamp - curr_timestamp);
032 |     let reward_growth_delta = checked_mul_div(
033 |         time_delta,
034 |         reward_info.emissions_per_second_x64,
035 |         whirlpool.liquidity,
036 |     )
037 |     .unwrap_or(0);
040 |     let curr_growth_global = reward_info.growth_global_x64;
041 |     reward_info.growth_global_x64 = curr_growth_global.wrapping_add(reward_growth_delta);

/* programs/whirlpool/src/state/whirlpool.rs */
190 | pub fn update_rewards(
191 |     &mut self,
192 |     reward_infos: [WhirlpoolRewardInfo; NUM_REWARDS],
193 |     reward_last_updated_timestamp: u64,
194 | ) {
195 |     self.reward_last_updated_timestamp = reward_last_updated_timestamp;
```

```
196 |     self.reward_infos = reward_infos;  
197 | }
```

Consequently, twenty one-second updates can accrue less than one twenty-second update with identical emissions and liquidity. The practical impacts depend on pool parameters. The loss can accumulate for liquidity providers but does not directly cause a protocol loss.

Resolution

The team acknowledged the behavior and accepted it as-is.

They explained that rounding in the owed calculation occurs in whole `u64` reward base units, while frequent timestamp updates round at the `u128` Q64 growth level, making the impact significantly smaller.

In addition, swaps advance `reward_last_updated_timestamp`. So popular pools, which they expect to be more likely to offer rewards, already update frequently through normal trading, without deliberate grieving.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Orca Management Company, S.A (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

